

Atividade Prática Avaliativa:

Engenharia de Dados e Escalabilidade

Objetivo: Implementar a lógica de processamento de um sistema de recomendação real usando **Matrizes Esparsas** para otimizar memória e desempenho.

1. Big Data na Recomendação (Responda como //comentários)

Uma matriz esparsa é aquela onde a maioria dos elementos são zeros. No Netflix, temos uma matriz onde as **linhas são usuários** e as **colunas são filmes**. Como um usuário assiste a apenas uma fração minúscula do catálogo, armazenar todos os zeros é um desperdício massivo de recursos.

Desafio de Cálculo de Memória

Considere o seguinte cenário real:

- **Usuários:** 1.000.000 (1 milhão)
- **Filmes no Catálogo:** 10.000
- **Média de filmes assistidos por usuário:** 100

Sua tarefa (As respostas de cada exercício, devem estar como comentários no arquivo “.JS” dessa tarefa:

1. Calcule quantos **Gigabytes (GB)** seriam necessários para armazenar essa matriz no formato **Denso** (onde guardamos todos os zeros), considerando que cada número ocupa o tipo padrão de **4 bytes**.
 2. Calcule o espaço necessário no formato **Esparso (COO)**, onde para cada filme assistido guardamos a tripla: {linha, coluna, valor}. (Dica: cada tripla terá 3 números de 4 bytes cada).
 3. Qual é a economia real de memória em porcentagem?
-

2. Big Data na Netflix (Código)

Imagine que você é um desenvolvedor na Netflix. Temos uma matriz onde as linhas são usuários e as colunas são filmes. Como a maioria dos usuários não assistiu a todos os filmes, usamos o formato COO (Lista de Triplas) para não desperdiçar memória.

Especificações Técnicas:

Total de Usuários (Linhas): 4

Total de Filmes (Colunas): 5

Vetor de Pesos: Contém a importância de cada um dos 5 filmes para uma recomendação atual.

Sua Tarefa de Programação

Implemente em JavaScript uma função que realize a multiplicação da Matriz Esparsa (COO) pelo Vetor de Pesos.

Requisitos do Código:

A função deve receber um array de objetos (a matriz) e um array simples (o vetor).

Proibido: Você não deve criar uma matriz densa (com zeros) em nenhum momento.

O processamento deve ser eficiente, percorrendo apenas os elementos que existem na lista de triplas.

Template para implementação:

JavaScript

```
/**
 * @param {Array} matrizEsparsa - Lista de objetos {linha, coluna, valor}
 * @param {Array} vetorDenso - Array com os pesos dos filmes
 */
function multiplicarRecomendacao(matrizEsparsa, vetorDenso) {
    // 1. Crie o vetor de resultado com tamanho 4 (número de usuários) preenchido com zeros
    // Dica: Use new Array(x).fill(x)

    // 2. Percorra a matriz esparsa (lista de triplas) com um único laço

    // 3. Aplique a lógica: Multiplique o 'valor' da nota pelo peso no 'vetorDenso'
    // correspondente à 'coluna' e some no índice 'linha' do resultado.

    // 4. Retorne o vetor de scores final
}

// --- DADOS PARA TESTE ---
const avaliacoes = [
    // Usuário 0 -> Filme 1 (Nota 5)
    // Usuário 1 -> Filme 3 (Nota 2)
    // Usuário 3 -> Filme 0 (Nota 4)
];

const pesos = [10, 20, 30, 40, 50]; // Pesos para os filmes 0, 1, 2, 3 e 4

Resultado esperado: [100, 80, 0, 40, 0]
console.log("Seu resultado:      ", multiplicarRecomendacao(avaliacoes, pesos));
```

3. Entrega

- O arquivo .js com a função implementada e o teste funcionando.
- As respostas das questões abertas devem estar como //comentários no arquivo .js
- Um breve parágrafo justificando: por que este algoritmo é mais rápido do que um que percorre a matriz linha por linha e coluna por coluna?